

Tangle Blueprint SDK

Developer Guide

Building Decentralized Services with Rust

Tangle Foundation

June 2026

Abstract

The Tangle Blueprint SDK provides the tooling for building decentralized services that interact with the Tangle protocol. This guide covers the SDK's architecture, design patterns, and the path from concept to deployed service. Written for developers familiar with Rust and blockchain concepts, it presents practical examples ranging from simple handlers to production-grade services including oracles, AI inference, keepers, and threshold cryptography.

Contents

1	Introduction	2
1.1	What You'll Build	2
1.2	Why Rust	2
2	Architecture Overview	2
2.1	Core Components	2
2.2	Execution Flow	3
3	Your First Blueprint	3
3.1	Project Setup	3
3.2	The Complete Example	3
3.3	Understanding the Code	4
4	The Hook System	4
4.1	Lifecycle Hooks	4
4.2	Blueprint Extensions	6
4.3	Slashing Customization	6
4.4	Membership Customization	6
4.5	Job Configuration	6
4.6	Hook Versioning (MBSM)	7
5	Triggers in Depth	7
5.1	Event Triggers	7
5.2	Cron Triggers	7
5.3	Webhook Triggers	7
5.4	x402 Paid Triggers	7
5.5	Custom Triggers (BackgroundKeeper)	8

6	BLS Aggregation	9
6.1	When to Use Aggregation	9
6.2	Aggregation Flow	9
6.3	Configuration	9
7	Use Case Examples	9
7.1	Price Oracle Service	9
7.2	AI Inference Service	10
7.3	Keeper Service	10
7.4	Threshold Signature Service	11
7.5	Pattern Selection Guide	12
8	Error Handling and Reliability	12
8.1	Handler Errors	12
8.2	Submission Failures	12
8.3	Chain Reorganizations	13
8.4	Crash Recovery	13
8.5	Graceful Shutdown	13
9	Quality of Service	13
9.1	QoS Architecture	13
9.2	Built-in Metrics	14
9.3	QoS-Based Slashing	14
9.3.1	Example: Heartbeat-Based Slashing	14
9.3.2	Example: Latency-Based Slashing	15
9.3.3	Example: Availability-Based Slashing	15
9.4	Connecting SDK to On-Chain Slashing	16
9.5	Design Considerations	17
10	Verifiability and Detection	17
10.1	Verification Economics	17
10.2	Verification Approaches	18
10.3	AI-Specific Challenges	18
10.4	Worked Example: Verifiable AI Sandbox	18
10.5	Worked Example: Verifiable Oracle	19
10.6	Formal Verification Properties	20
11	Working with the Marketplace	20
11.1	Payment Models	20
11.2	The RFQ System	20
11.3	Off-Chain Coordination	21
11.4	Failure Modes	21
11.5	Quote Serving Architecture	21
11.6	Price Discovery	21
12	Testing and Development	21
12.1	Local Testing	21
12.2	Integration Testing	22
12.3	Testnet Deployment	22
12.4	Observability	22

13 P2P Networking	22
13.1 Network Architecture	22
13.2 Discovery Mechanisms	23
13.3 Messaging Protocols	23
13.4 Authentication and Security	23
13.5 Network Events	23
13.6 BLS Signature Aggregation	24
13.7 Custom Protocols	24
14 Deployment Checklist	24
15 Operator Configuration	24
15.1 Core Settings	24
15.2 Networking Settings	25
15.3 Example Configuration	25
15.4 Keystore Setup	25
16 Resources	25

1 Introduction

Tangle enables developers to define computational services that run across a network of independent operators. The Blueprint SDK is the primary toolkit for building these services in Rust.

This guide assumes familiarity with:

- Rust programming and `async/await` patterns
- Basic blockchain concepts (transactions, events, contracts)
- Cryptographic primitives (signatures, hashes)

1.1 What You'll Build

A blueprint defines a service type. When deployed, operators register to provide that service, and customers create service instances by selecting operators and configuring parameters. Your blueprint code runs on operator infrastructure, processing jobs and submitting results.

1.2 Why Rust

The SDK is written in Rust for several reasons:

- **Performance:** Critical for operators processing high-throughput workloads. Rust compiles to native code with zero-cost abstractions.
- **Safety:** Prevents memory errors and race conditions. For operator software handling valuable stake and sensitive data, safety is non-negotiable.
- **Async runtime:** Tokio provides efficient concurrent execution for handling blockchain events, P2P networking, and customer requests simultaneously.
- **WebAssembly compilation:** Enables portable execution for sandboxed environments.
- **Ecosystem compatibility:** Excellent integration with blockchain tooling (alloy, libp2p, ark-bn254).

2 Architecture Overview

The SDK is organized into composable components.

2.1 Core Components

Producers generate job calls from external events:

- **TangleProducer** watches blockchain events
- **CronJob** generates calls on schedules
- Custom producers watch APIs, queues, or any event source

Router dispatches job calls to handlers. When a job arrives, the router examines the job index and invokes the corresponding function.

Handlers implement job-specific logic. Each handler receives context (extractors) and returns a result. Handlers are async functions.

Consumer submits results to the blockchain. For aggregated results, the consumer coordinates with other operators to collect signatures.

Background services run alongside job processing:

- `BackgroundService` for general long-running tasks
- `BackgroundKeeper` for Tangle lifecycle automation

P2P layer provides inter-operator communication for quote dissemination, signature aggregation, and custom coordination.

2.2 Execution Flow

1. Producer detects event (blockchain, cron, custom)
2. Producer generates job call
3. Router dispatches to appropriate handler
4. Handler executes and returns result
5. Consumer submits result (coordinating aggregation if needed)

Throughout this flow, background services continue running and the P2P layer handles coordination.

3 Your First Blueprint

Let's build a minimal blueprint that squares numbers.

3.1 Project Setup

Create a new Rust project:

```
cargo new my-blueprint
cd my-blueprint
```

Add dependencies to `Cargo.toml`:

```
[dependencies]
blueprint-sdk = { git = "https://github.com/tangle-network/blueprint",
  branch = "main", features = ["tangle"] }
tokio = { version = "1", features = ["full"] }
```

3.2 The Complete Example

```
use blueprint_sdk::tangle::extract::{TangleArg, TangleResult};
use blueprint_sdk::tangle::{TangleConsumer, TangleProducer, TangleLayer};
use blueprint_sdk::contexts::tangle::TangleClientContext;
use blueprint_sdk::runner::{BlueprintRunner, config::BlueprintEnvironment};
use blueprint_sdk::runner::tangle::TangleConfig;
use blueprint_sdk::Router;

pub const SQUARE_JOB: u8 = 0;

// Job function: extracts input, returns wrapped result
pub async fn square(TangleArg(x): TangleArg<u64>) -> TangleResult<u64> {
    {
        TangleResult(x * x)
    }
}

pub fn router() -> Router {
    Router::new().route(SQUARE_JOB, square.layer(TangleLayer))
}
```

```

}

#[tokio::main]
async fn main() -> Result<(), blueprint_sdk::Error> {
    let env = BlueprintEnvironment::load()?;
    let client = env.tangle_client().await?;
    let service_id = env.protocol_settings.tangle()?.service_id.unwrap();

    BlueprintRunner::builder(TangleConfig::default(), env)
        .router(router())
        .producer(TangleProducer::new(client.clone(), service_id))
        .consumer(TangleConsumer::new(client))
        .run()
        .await
}

```

3.3 Understanding the Code

The SDK uses an **extractor pattern** inspired by web frameworks:

- **TangleArg<T>** extracts ABI-encoded inputs from job calls
- **TangleResult<T>** wraps outputs for ABI-encoded submission
- **TangleLayer** handles encoding/decoding transformation

Job functions are plain async functions. The main function composes SDK components: producer watches for jobs, router directs to handlers, consumer submits results.

4 The Hook System

Blueprints customize protocol behavior through hooks, functions called at specific lifecycle points.

4.1 Lifecycle Hooks

onRegister validates operator registrations. A blueprint for AI services might require operators to prove GPU capability.

onRequest validates service requests. A membership service might verify the customer's eligibility.

onApprove processes operator approvals. A DeFi service might lock collateral or initialize state.

onUnappliedSlash executes custom slashing logic when a slash is finalized. A threshold service might redistribute key shares. The **slashPercent** argument is the dispute-survived, per-service-capped slash as a whole percent (0–100); note that **proposeSlash** takes the proposed amount in basis points while BSM callbacks such as this one receive a whole percent (a deliberately different unit, not a bug).

onServiceTermination handles cleanup. A subscription service might finalize billing.

Hooks are implemented in Solidity by extending **BlueprintServiceManagerBase**:

```
contract MyBlueprintBSM is BlueprintServiceManagerBase {
    // Track registered operators
    mapping(address => bool) public isRegistered;
    mapping(address => bytes) public operatorMetadata;

    // Validate operator registration
    function onRegister(
        address operator,
        bytes calldata inputs
    ) external payable override onlyFromTangle {
        // Decode and validate registration inputs
        (uint256 gpuMemoryGB, string memory region) =
            abi.decode(inputs, (uint256, string));

        require(gpuMemoryGB >= 24, "Minimum 24GB GPU required");
        require(bytes(region).length > 0, "Region required");

        isRegistered[operator] = true;
        operatorMetadata[operator] = inputs;

        emit OperatorRegistered(operator, gpuMemoryGB, region);
    }

    // Process service requests
    function onRequest(
        uint64 requestId,
        address requester,
        address[] calldata operators,
        bytes calldata requestInputs,
        uint64 ttl,
        address paymentAsset,
        uint256 paymentAmount
    ) external payable override onlyFromTangle {
        // Validate the request parameters
        require(operators.length >= 3, "Minimum 3 operators");
        require(ttl >= 1 days, "Minimum 1 day TTL");

        // Store request for later reference
        requests[requestId] = RequestData({
            requester: requester,
            operatorCount: operators.length,
            ttl: ttl
        });
    }

    // Handle slashing events
    function onUnappliedSlash(
        uint64 serviceId,
        bytes calldata offender,
        uint8 slashPercent
    ) external override onlyFromTangle {
        // Custom logic: redistribute key shares if threshold service
        address operator = abi.decode(offender, (address));
        emit OperatorSlashed(serviceId, operator, slashPercent);

        // Trigger key resharing protocol
        if (slashPercent >= 50) {

```

```
        initiateKeyReshare(serviceId, operator);
    }
}
}
```

4.2 Blueprint Extensions

The SDK provides base contracts that extend `BlueprintServiceManagerBase` with reusable economic mechanics:

TokenizedBlueprintBase enables blueprints to issue community tokens tied to their service. Operators and users can stake these tokens to earn Synthetix-style staking rewards, creating aligned incentives around blueprint adoption. The base contract handles token issuance, staking accounting, and reward distribution.

BuybackBlueprintBase adds AMM buyback mechanics. A portion of service fees is automatically used to buy back the blueprint's community token on decentralized exchanges, creating sustained demand. Blueprint developers configure the buyback percentage and target AMM pool. The buyback is funded from the blueprint developer's share of the service fee split (see Payment Models); it does not draw from operator, staker, or protocol shares.

These extensions compose with each other and with custom hooks, enabling blueprints to build token economies around their services. They are opt-in base contracts: a blueprint must explicitly extend and deploy them, and they are not auto-deployed by the protocol. Treat them as early-maturity primitives. On-chain buybacks execute against public AMM pools and are therefore MEV-exposed (front-running, sandwiching); developers should bound slippage, consider TWAP execution, and size buybacks conservatively relative to pool depth.

4.3 Slashing Customization

Blueprints define their own slashing conditions:

`querySlashingOrigin` returns a *single* authorized address that may propose slashes (in addition to the service owner and blueprint owner). There is no built-in permissionless "anyone with evidence" path: to support open, evidence-based proposals, point `querySlashingOrigin` at a verification contract you deploy that validates submitted evidence and forwards `proposeSlash` on the caller's behalf.

`queryDisputeOrigin` specifies who may dispute (the accused operator, governance, delegated arbitrators).

Custom verification contracts implement evidence validation, dispute resolution, and penalty calculation.

4.4 Membership Customization

Membership is configured on the service request, not through blueprint hooks. A request specifies a `Types.MembershipModel` (`Fixed` or `Dynamic`) together with `minOperators`, and the blueprint constrains which models are permitted via its `supportedMemberships` allow-list. There are no `queryMinOperators`, `queryMaxOperators`, or `queryExitDelay` hooks; operator exit timing is governed by the protocol's staking unbonding schedule (see Staking), not by a blueprint callback.

Within these models, custom `onRegister`/`onRequest` logic can still implement reputation-based entry, stake-weighted selection, or geographic distribution requirements.

4.5 Job Configuration

`requiresAggregation` determines whether results need multi-operator consensus.

`getAggregationThreshold(serviceId, jobIndex)` returns (uint16 thresholdBps, uint8 mode) per job: thresholdBps is the required agreement in basis points and mode selects how it is counted (0 = count-based, 1 = stake-weighted). The protocol default is 6700/count-based (two-thirds by operator count).

4.6 Hook Versioning (MBSM)

The protocol routes hook calls into a blueprint's service manager through the Master Blueprint Service Manager (MBSM). The MBSMRegistry tracks MBSM versions, and each blueprint is pinned to the MBSM version it was created against. The set and ABI of lifecycle hooks (onRegister, onRequest, onApprove, onUnappliedSlash, onServiceTermination, and the query hooks) are therefore *version-bound*: a given MBSM version defines exactly which hooks exist and their signatures. When adopting a newer MBSM version, recheck the hook ABIs against that version rather than assuming forward compatibility.

5 Triggers in Depth

Triggers initiate job execution. The SDK provides multiple trigger types.

5.1 Event Triggers

TangleProducer watches blockchain events via standard EVM JSON-RPC:

- Queries Tangle contract logs
- Filters for the configured service ID
- Extracts job parameters and metadata
- Retries transient RPC failures with backoff

5.2 Cron Triggers

CronJob executes on schedules using standard cron expressions with seconds precision:

```
// Every second
"* * * * *"

// Every 5 minutes
"0 */5 * * * *"

// Daily at midnight
"0 0 0 * * *"
```

5.3 Webhook Triggers

When your service needs to react to external systems (monitoring, exchanges, schedulers), operators can expose HTTP endpoints that translate inbound requests into job executions. This is implemented as a gateway (HTTP server) plus a producer that injects JobCalls into the runner.

5.4 x402 Paid Triggers

For HTTP-first services that want stablecoin settlement on EVM chains, operators can optionally accept paid HTTP job calls via the x402 protocol. After payment is verified and settled, the request is translated into a job execution.

Cron triggers suit periodic tasks: heartbeats, maintenance, report generation, API polling.

5.5 Custom Triggers (BackgroundKeeper)

For lifecycle automation, implement **BackgroundKeeper**. The SDK provides built-in keepers:

- **EpochKeeper** for epoch-boundary reward distribution (treasury-funded emission paid out from the pre-funded **InflationPool**)
- **StreamKeeper** for payment stream settlement
- **RoundKeeper** for round-based protocols

TNT is hard-capped at $\text{MAX_SUPPLY} = 109,255,636.92$ TNT, and the full cap is minted at genesis across four buckets (Substrate, EVM, Treasury, Foundation), so **totalSupply** equals the cap from block zero. The **InflationPool** cannot mint; it only distributes TNT it already holds. “Inflation” here is treasury-funded *emission*: roughly 1% of supply per year ($\approx 1,092,556$ TNT) is transferred from the Treasury bucket into the **InflationPool** and paid out at epoch boundaries. This dilutes no one and creates no new supply. A governance **MINTER_ROLE** exists and can mint only up to the cap, but with genesis already at the cap it has no headroom (a **mint()** that would exceed the cap reverts).

```
use blueprint_sdk::tangle::services::{BackgroundKeeper, KeeperConfig,
KeeperHandle};
use tokio::sync::broadcast;

struct PriceMonitorKeeper;

impl BackgroundKeeper for PriceMonitorKeeper {
    const NAME: &'static str = "price-monitor";

    fn start(config: KeeperConfig, mut shutdown: broadcast::Receiver<()>)
        -> KeeperHandle
    {
        let handle = tokio::spawn(async move {
            loop {
                tokio::select! {
                    _ = shutdown.recv() => break,
                    result = Self::check_and_execute(&config) => {
                        if let Err(e) = result {
                            tracing::warn!("Check failed: {e}");
                        }
                    }
                }
                tokio::time::sleep(config.round_check_interval).await;
            }
            Ok(())
        });
        KeeperHandle { handle, name: Self::NAME }
    }

    async fn check_and_execute(config: &KeeperConfig)
        -> blueprint_sdk::tangle::services::KeeperResult<bool>
    {
        // Monitor conditions and trigger actions
        Ok(false)
    }
}
```

6 BLS Aggregation

Multi-operator services use BLS signature aggregation for efficient consensus.

6.1 When to Use Aggregation

Use aggregation when:

- Multiple operators must agree on a result
- You need Byzantine fault tolerance
- Verification cost matters (BLS aggregates cheaply)

Skip aggregation when:

- Speed is paramount (first-to-submit wins)
- Single-operator services
- Outputs are independently verifiable

6.2 Aggregation Flow

1. Each operator computes result and signs with BLS key
2. Operators broadcast signatures via P2P
3. Aggregator collects signatures until threshold met
4. Aggregated signature submitted with combined result
5. Contract verifies aggregate signature on-chain

6.3 Configuration

The service manager contract controls aggregation:

- `requiresAggregation(serviceId, jobIndex)` returns whether job needs aggregation
- `getAggregationThreshold(serviceId, jobIndex)` returns `(uint16 thresholdBps, uint8 mode)` (default 6700/count-based)

7 Use Case Examples

7.1 Price Oracle Service

A price oracle provides asset prices to DeFi protocols.

```
use blueprint_sdk::tangle::extract::{TangleArg, TangleResult};
use std::time::{SystemTime, UNIX_EPOCH};

pub const FETCH_PRICE_JOB: u8 = 0;

pub async fn fetch_price(
    TangleArg(pair): TangleArg<AssetPair>
) -> TangleResult<PriceData> {
    // Fetch from multiple sources for redundancy
    let binance = fetch_binance(&pair).await.unwrap_or_default();
    let coinbase = fetch_coinbase(&pair).await.unwrap_or_default();
    let chainlink = fetch_chainlink(&pair).await.unwrap_or_default();

    // Median aggregation for manipulation resistance
    let price = median(vec![binance, coinbase, chainlink]);
    let timestamp = SystemTime::now();
}
```

```

        .duration_since(UNIX_EPOCH).unwrap().as_secs();

    TangleResult(PriceData { pair, price, timestamp, sources: 3 })
}

```

Triggers: Event (price requests) + Cron (periodic updates)

Submission: Aggregated (median across operators)

Verification: Cross-operator comparison; deviations trigger review

7.2 AI Inference Service

An AI inference service runs language models for customers.

```

use blueprint_sdk::tangle::extract::{TangleArg, TangleResult};
use blueprint_sdk::extract::Context;

pub const INFERENCE_JOB: u8 = 0;

pub async fn inference(
    Context(config): Context<InferenceConfig>,
    TangleArg(request): TangleArg<InferenceRequest>,
) -> TangleResult<Completion> {
    let model = ModelClient::new(&config.model_endpoint);
    let completion = model.complete(
        &request.prompt,
        request.max_tokens,
        request.temperature,
    ).await?;

    TangleResult(Completion {
        text: completion.text,
        usage: completion.tokens_used,
        model: config.model_id.clone(),
    })
}

```

Triggers: Event (inference requests)

Submission: Direct or aggregated depending on requirements

Verification: Model fingerprinting challenges detect substitution

7.3 Keeper Service

A keeper monitors on-chain conditions and executes transactions.

```

use blueprint_sdk::runner::{BackgroundService, error::RunnerError};
use alloy_primitives::Address;
use std::time::Duration;
use tokio::sync::oneshot::Receiver;

pub struct LiquidationService {
    lending_protocol: Address,
    check_interval: Duration,
}

impl BackgroundService for LiquidationService {
    async fn start(&self) -> Result<Receiver<Result<(), RunnerError>>,
        RunnerError> {
        let (tx, rx) = oneshot::channel();
    }
}

```

```

    let protocol = self.lending_protocol;
    let interval = self.check_interval;

    tokio::spawn(async move {
        loop {
            match fetch_undercollateralized(&protocol).await {
                Ok(positions) => {
                    for position in positions {
                        if let Err(e) = execute_liquidation(&
                            position).await {
                            tracing::warn!("Liquidation failed: {e}");
                        }
                    }
                }
                Err(e) => tracing::warn!("Failed to fetch: {e}"),
            }
            tokio::time::sleep(interval).await;
        }
    });
    Ok(rx)
}
}

```

Triggers: BackgroundKeeper (condition monitoring)

Submission: Direct (speed-critical)

Verification: Transaction success is self-evident

7.4 Threshold Signature Service

A threshold service generates signatures without any party holding the complete key.

```

use blueprint_sdk::tangle::extract::{TangleArg, TangleResult};
use blueprint_sdk::extract::Context;

pub const SIGN_JOB: u8 = 0;

pub async fn sign(
    Context(state): Context<ThresholdState>,
    TangleArg(message): TangleArg<Bytes>,
) -> TangleResult<PartialSignature> {
    let key_share = state.dkg_share.as_ref()
        .ok_or_else(|| Error::Other("No DKG share".into()))?;
    let partial = key_share.partial_sign(&message)?;

    TangleResult(PartialSignature {
        index: state.operator_index,
        signature: partial,
    })
}

```

Triggers: Event (signing requests)

Submission: Aggregated (threshold combination)

Verification: Signature validity is cryptographically verifiable

7.5 Pattern Selection Guide

Service Type	Triggers	Submission
Oracle	Event + Cron	Aggregated (median)
AI Inference	Event	Direct or Aggregated
Keeper	BackgroundKeeper	Direct (speed-critical)
Threshold Crypto	Event	Aggregated (threshold)
Monitoring	Cron	Direct

8 Error Handling and Reliability

Production software must handle failures gracefully. The SDK provides structured error types and recovery patterns.

8.1 Handler Errors

Handler errors are caught and logged. A handler returning `Err` does not crash the operator. Depending on blueprint design, the operator may skip submission or submit an error indicator.

```
use blueprint_sdk::tangle::extract::{TangleArg, TangleResult};
use thiserror::Error;

#[derive(Error, Debug)]
pub enum JobError {
    #[error("External API unavailable: {0}")]
    ApiUnavailable(String),
    #[error("Invalid input: {0}")]
    InvalidInput(String),
    #[error("Computation timeout")]
    Timeout,
}

pub async fn fetch_price(
    TangleArg(pair): TangleArg<AssetPair>
) -> Result<TangleResult<PriceData>, JobError> {
    // Validate input
    if pair.base.is_empty() || pair.quote.is_empty() {
        return Err(JobError::InvalidInput("Empty asset pair".into()));
    }

    // Fetch with timeout
    let price = tokio::time::timeout(
        Duration::from_secs(10),
        fetch_from_sources(&pair)
    ).await.map_err(|_| JobError::Timeout)?;

    Ok(TangleResult(price))
}
```

8.2 Submission Failures

Operators should assume transient RPC failures during result submission (rate limits, timeouts, nonce conflicts, temporary insufficient funds). The SDK surfaces submission errors to the runner. In practice, operators run blueprints under a supervisor (systemd, Kubernetes) and apply retries at the process or RPC layer.

```
let client = env.tangle_client().await?;
let service_id = env.protocol_settings.tangle()?.service_id.unwrap();

let producer = TangleProducer::new(client.clone(), service_id);

let consumer = TangleConsumer::new(client);

BlueprintRunner::builder(TangleConfig::default(), env)
    .router(router())
    .producer(producer)
    .consumer(consumer)
    .run()
    .await
```

8.3 Chain Reorganizations

EVM chains can reorganize. If your service requires stricter finality, consider polling behind the head block, or reprocessing a small block window on restart. Job handlers should be idempotent where possible.

8.4 Crash Recovery

The simplest recovery strategy is to restart the process and resume from a known block height. The producer supports starting from a specific block; operators can persist the last processed block (or a conservative checkpoint) in their own storage.

```
let client = env.tangle_client().await?;
let service_id = env.protocol_settings.tangle()?.service_id.unwrap();

// Resume from a stored checkpoint (or start from genesis)
let start_block: u64 = load_checkpoint().unwrap_or(0);

BlueprintRunner::builder(TangleConfig::default(), env)
    .router(router())
    .producer(TangleProducer::from_block(client.clone(), service_id,
        start_block))
    .consumer(TangleConsumer::new(client))
    .run()
    .await
```

8.5 Graceful Shutdown

On shutdown, operators typically rely on their process supervisor and implement application-level draining when needed. Job handlers should be cancellation-safe and side-effect aware.

9 Quality of Service

The Quality of Service (QoS) package provides monitoring, metrics, and observability for operator services.

9.1 QoS Architecture

The QoS system integrates multiple monitoring backends:

- **Prometheus:** Metrics collection via `/metrics` endpoint
- **OpenTelemetry:** Exports metrics to OTLP-compatible backends (Grafana Cloud, Data-dog)
- **Loki:** Structured logging with labels for log querying
- **Grafana:** Dashboards showing throughput, latency, errors, and resources

9.2 Built-in Metrics

- **System metrics:** CPU utilization, memory, disk I/O, network traffic
- **Blueprint metrics:** Jobs executed, execution times, success/failure rates, queue depths
- **Blueprint status:** Uptime, last heartbeat, status codes, status messages

9.3 QoS-Based Slashing

QoS provides data; slashing provides consequences. Connecting these systems enables automatic SLA enforcement.

Architecture: QoS metrics flow from operators to a monitoring service. When metrics violate thresholds, the monitor proposes a slash via Tangle. The service manager's `querySlashingOrigin` hook authorizes the monitor. After the dispute window, slashing executes.

The on-chain entry point is `proposeSlash(uint64 serviceId, address operator, uint16 slashBps, bytes32 evidence)`. The amount is in *basis points*, the call *reverts on zero evidence*, the effective slash is capped at the service's `maxSlashBps` (5000 = 50% at mainnet launch), and the realized penalty is scaled by the operator's committed exposure to the service. The examples below pass a non-zero `evidence` hash committing to the off-chain measurement.

9.3.1 Example: Heartbeat-Based Slashing

```
contract HeartbeatSlashingManager {
    uint256 public constant MAX_MISSED = 3;
    uint256 public constant INTERVAL = 5 minutes;

    mapping(uint64 => mapping(address => uint256)) public lastHeartbeat;
    mapping(uint64 => mapping(address => uint256)) public missedCount;

    function recordHeartbeat(uint64 serviceId) external {
        require(isOperatorInService(serviceId, msg.sender));
        lastHeartbeat[serviceId][msg.sender] = block.timestamp;
        missedCount[serviceId][msg.sender] = 0;
    }

    function checkAndSlash(uint64 serviceId, address operator) external
    {
        uint256 elapsed = block.timestamp - lastHeartbeat[serviceId][operator];
        uint256 missed = elapsed / INTERVAL;

        if (missed > MAX_MISSED) {
            bytes32 evidence = keccak256(
                abi.encode(serviceId, operator, missed, block.timestamp)
            );
        }
    }
}
```



```

        // slashBps in basis points (500 = 5%); zero evidence
        reverts
        tangle.proposeSlash(serviceId, operator, 500, evidence);
    }
}

function querySlashingOrigin(uint64) external view returns (address
) {
    return address(this);
}
}

```

9.3.2 Example: Latency-Based Slashing

```

contract LatencySlashingManager {
    uint256 public constant MAX_LATENCY_MS = 5000; // 5 second SLA
    uint256 public constant VIOLATION_THRESHOLD = 10;

    address public latencyOracle;
    mapping(uint64 => mapping(address => uint256)) public violations;

    function reportLatencyViolation(
        uint64 serviceId,
        address operator,
        uint256 actualLatencyMs,
        bytes calldata proof
    ) external {
        require(msg.sender == latencyOracle);
        require(actualLatencyMs > MAX_LATENCY_MS);

        violations[serviceId][operator]++;

        if (violations[serviceId][operator] >= VIOLATION_THRESHOLD) {
            bytes32 evidence = keccak256(
                abi.encode(serviceId, operator, actualLatencyMs, proof)
            );
            // slashBps in basis points (1000 = 10%); zero evidence
            reverts
            tangle.proposeSlash(serviceId, operator, 1000, evidence);
            violations[serviceId][operator] = 0;
        }
    }
}

```

9.3.3 Example: Availability-Based Slashing

```

contract AvailabilitySlashingManager {
    uint256 public constant MIN_UPTIME_PERCENT = 99;
    uint256 public constant WINDOW = 1 days;

    struct UptimeRecord {
        uint256 windowStart;
        uint256 successfulChecks;
        uint256 totalChecks;
    }

    mapping(uint64 => mapping(address => UptimeRecord)) public uptime;
}

```

```

function recordCheck(uint64 serviceId, address operator, bool
    available)
    external onlyMonitor
{
    UptimeRecord storage r = uptime[serviceId][operator];

    if (block.timestamp > r.windowStart + WINDOW) {
        if (r.totalChecks > 0) {
            uint256 pct = (r.successfulChecks * 100) / r.
                totalChecks;
            if (pct < MIN_UPTIME_PERCENT) {
                // Liveness penalty is a small band: 10 bps per
                // percentage point, capped at 200 bps (2%).
                // Liveness
                // faults are not equivocation; large slashes are
                // reserved
                // for safety violations. Any value above the
                // service's
                // maxSlashBps is further capped by the protocol on
                // execute.
                uint256 slashBps = (MIN_UPTIME_PERCENT - pct) * 10;
                if (slashBps > 200) slashBps = 200;
                bytes32 evidence = keccak256(
                    abi.encode(serviceId, operator, pct, r.
                        windowStart)
                );
                tangle.proposeSlash(
                    serviceId, operator, uint16(slashBps), evidence
                );
            }
        }
        r.windowStart = block.timestamp;
        r.successfulChecks = 0;
        r.totalChecks = 0;
    }

    r.totalChecks++;
    if (available) r.successfulChecks++;
}
}

```

9.4 Connecting SDK to On-Chain Slashing

The full flow:

```

// 1. Operator runs QoS-enabled service
let qos = QoSService::new(qos_config).await?;
qos.start_collection().await?;

// 2. Off-chain monitor queries metrics
// GET http://operator:9090/metrics

// 3. Monitor detects SLA violation
if avg_latency > SLA_THRESHOLD {
    // 4. Monitor calls service manager
    service_manager.reportLatencyViolation(

```

```

        service_id, operator, latency_ms, proof
    ).send().await?;
}
// 5. Service manager proposes slash
// 6. After dispute window, slash executes

```

9.5 Design Considerations

When designing QoS-based slashing:

- Calibrate thresholds to distinguish misbehavior from normal variance
- Ensure measurement integrity through multiple monitors or proofs
- Include evidence in proposals for dispute resolution
- Make penalties proportional to harm
- Consider grace periods for new operators

Dispute-game guardrails: at mainnet launch a slash proposal opens a dispute that requires a `disputeBond` of 0.02 ETH (refunded if the dispute is cancelled, forfeited if the slash executes). The `disputeResolutionDeadline` is 21 days and is *fail-open*: if the dispute is not resolved in time the slash auto-executes. Because resolution is fail-open, blueprints relying on QoS slashing should run an *m-of-n* set of independent monitors so a single offline monitor cannot block a justified slash, and conversely so a single faulty monitor cannot force one. An operator with any pending slash is frozen from unstaking, which prevents stake-exit during the dispute window.

10 Verifiability and Detection

Slashing requires detection. This section covers verification strategies, worked examples, and formal properties.

10.1 Verification Economics

This section uses a few economic terms defined fully in the Tangle economics paper, summarized here: *PfC* (profit-from-corruption) is the most an attacker can extract by violating the service; *CoC* (cost-of-corruption) is the stake an attacker must put at risk to do so; *exposure* is the fraction of an operator's stake committed to a given service and thus slashable for it; and the *security ratio* is CoC/PfC . These are nominal upper bounds; the realized figures depend on how reliably violations are detected and slashed.

Not all verification is worthwhile. Let C_v be verification cost, P_d detection probability, S slash amount, and P_c probability of cheating. Verification is justified when:

$$C_v < P_d \times S \times P_c$$

This reveals design principles. High slashes enable profitable verification even with moderate detection. Probabilistic verification (random sampling) achieves deterrence at lower cost. Verification costs should scale with stakes.

Slashing realization. The slash amount S in these bounds is *nominal*. The amount actually lost by a corrupt operator is closer to $P_{\text{execute}} \times P_d \times S_{\text{nominal}}$, where P_{execute} accounts for the dispute game: at launch the dispute window is fail-open over 21 days, but a sophisticated attacker may still attempt to exit stake before execution (mitigated, not eliminated, by the pending-slash unstake freeze). Treat the headline CoC and security ratio as conditional bounds that hold only under the stated detection assumptions (probe-indistinguishability: challenges

are statistically indistinguishable from real jobs) and collusion assumptions (no coalition above the corruption threshold).

Deterrence versus the verifier’s private incentive. The condition $P_d \times S > V$ above is a *social* deterrence condition: it tells you whether cheating is unprofitable in aggregate. It is distinct from whether any individual party is privately paid enough to *run* the verification — the verifier’s dilemma. Detection has a real cost C_v , yet slash proceeds are routed by the protocol fee split rather than handed to whoever reported the violation. To close this gap, blueprints should fund an explicit reporter bounty or lean on the protocol’s permissionless keeper rebate (see Payment Models) so that producing and submitting evidence is itself EV-positive.

10.2 Verification Approaches

Redundant Execution: Multiple operators compute independently; disagreements trigger investigation. Simple but expensive. Use commit-reveal to prevent copying.

Optimistic with Fraud Proofs: Assume honest, challenge suspicious results with cryptographic proofs. Efficient for verifiable computations.

Challenge-Response: Periodic challenges test operator honesty. Maintain database of (input, expected output) pairs indistinguishable from regular jobs. Cost-effective for ongoing services.

TEE Attestation: Trusted Execution Environments provide hardware-backed verification. Strong guarantees but requires TEE hardware.

Reputation and Sampling: Aggregate customer reports. Random sampling selects executions for intensive verification.

10.3 AI-Specific Challenges

AI outputs are often non-deterministic and subjective.

Model Identity: Different models have different output distributions. Fingerprinting probes produce distinctive outputs. Comparing operator distributions against known fingerprints identifies which model runs.

Performance: Claims about tokens/second or latency can be tested empirically. Continuous random sampling prevents inflated benchmarks during known tests.

Isolation: Isolation failures may leave no visible trace. Defense in depth (network policies, syscall filtering, process isolation) makes breaches unlikely. Audit logging and anomaly detection increase detection probability.

10.4 Worked Example: Verifiable AI Sandbox

A sandbox service promising agents run on a specified frontier model (the configured model, e.g. a named Claude, GPT, or Llama variant) with specified isolation.

Verification mechanisms:

- Model fingerprinting: Challenge database of (prompt, expected pattern) pairs distinguishing the promised model from alternatives
- Performance: Heartbeat jobs with latency requirements
- Isolation: gVisor/Firecracker required at registration; QoS monitors for anomalies

Slashing configuration (each subject to the per-service `maxSlashBps` ceiling, 50% at main-net launch):

- Model substitution: 50% (severe, violates core promise)
- Performance degradation: 5% (moderate, affects quality)
- Isolation breach: configured at 100%, but realized slash is capped at `maxSlashBps` (50% at launch) unless governance raises the per-service ceiling for an equivocation-class fault

Quantified economics. Customer runs trading agents managing \$200,000. PfC = \$200,000 (assets under management).

Service uses 3 operators, each staking \$100,000 with 80% exposure. Corrupting requires all 3 (any honest operator detects substitution via challenges, under the probe-indistinguishability and no-collusion-above-threshold assumptions). The *nominal* stake at risk is $3 \times \$100,000 \times 0.8 = \$240,000$. With the launch `maxSlashBps` of 50%, however, a single corruption event can burn at most 50% of each committed stake, so the *realizable* CoC for a one-shot attack is $3 \times \$100,000 \times 0.8 \times 0.5 = \$120,000$.

Security ratio against realizable CoC = $\$120,000 / \$200,000 = 0.6$, below 1 — the 50% cap means a single substitution event is not deterred by stake alone at these parameters. Restoring a ratio above 1 (the economics paper’s target band; the previously cited “1.5x” is illustrative, not a protocol constant) requires raising stake or operator count, reducing exposure to a smaller AUM cap per service, or governance-raising `maxSlashBps` for the equivocation-class substitution fault.

Detection probability. Model fingerprinting challenges submitted 10 times daily. Each detects substitution with 85% probability, *assuming probes are indistinguishable from real jobs*; an operator who can identify probes can pass them while cheating real traffic, collapsing P_d . Over a 7-day window:

$$P_d = 1 - (1 - 0.85)^{70} \approx 99.99\%$$

Under that assumption detection is near-certain, but the deterrent is still bounded by the realizable slash above: effective expected cost to the attacker is approximately $P_{\text{execute}} \times P_d \times \$120,000$, not the nominal \$240,000.

10.5 Worked Example: Verifiable Oracle

A price oracle promising freshness (5 min), accuracy (1%), and availability (99%).

Verification mechanisms:

- Source verification: Signed data from 3+ of 5 approved sources submitted with prices
- Cross-operator comparison: Multiple operators submit independently; deviations >0.5% flagged
- External auditing: Anyone compares oracle prices to public market data

Slashing configuration (realized slash capped at the per-service `maxSlashBps`, 50% at launch):

- Source forgery: configured at 100% (safety/equivocation class), realized at the `maxSlashBps` ceiling unless governance raises it for this service
- Price deviation: 10% per incident, accumulating up to the ceiling
- Availability failure: a small liveness band of ~10 bps (0.1%) per missed update, capped at 200 bps (2%) — liveness faults are not equivocation and should not approach the safety ceiling

Economics. Oracle serves DeFi with \$100M exposure. Manipulated prices enable \$10M arbitrage (PfC = \$10M).

5 operators each staking \$3M with 100% exposure. The nominal stake of a 3-of-5 coalition is \$9M, but with the 50% launch `maxSlashBps` the realizable one-shot CoC is $\$9M \times 0.5 = \$4.5M$.

Security ratio against realizable CoC = 0.45 — insufficient under the no-collusion-above-threshold assumption. The blueprint should increase stake requirements, raise the corruption threshold, reduce exposure to smaller protocols, or have governance raise `maxSlashBps` for the source-forgery (equivocation) fault.

10.6 Formal Verification Properties

Property V1 (Detection Soundness): If operator behaves honestly, verification flags them for slashing with probability at most ϵ (false positive rate).

Importance: False positives discourage participation. High false positive rate is unusable.

Property V2 (Detection Completeness): If operator violates specification in manner M , verification detects with probability at least p_M .

Importance: Detection probability determines effective cost of cheating.

Property V3 (Economic Sufficiency): For all violation types M with benefit V_M : $p_M \times S_M > V_M$, where S_M is the *realizable* slash, i.e. $S_M \leq \text{maxSlashBps} \times (\text{committed exposure})$.

Importance: Core security condition. Blueprints must configure detection and slashing to satisfy this. Note the hard ceiling: if V_M exceeds maxSlashBps times an operator's committed stake (50% at launch), no detection probability or slash schedule deters that operator alone — the only remedies are raising stake, increasing the number of operators required to corrupt, reducing per-service exposure, or governance-raising maxSlashBps for that equivocation-class fault.

Property V4 (Verification Efficiency): Verification cost $C_v < \alpha \times V_s$ where V_s is service value.

Importance: Verification costing more than service provides is irrational. Target $\alpha < 0.1$.

Blueprints should analyze mechanisms against these properties, documenting false positive rates, detection probabilities, and costs.

11 Working with the Marketplace

Blueprints interact with the Tangle marketplace for service discovery and pricing. Understanding this interaction helps developers design robust services.

11.1 Payment Models

A service selects one of four payment models at request time:

PayOnce a single up-front payment for the service term.

Subscription recurring billing. Billing is permissionless and TWAP-fair: anyone may call `billSubscription` when a period is due, and the call carries a 0.5% keeper rebate (carved from the protocol's share) so that running the keeper is EV-positive.

EventDriven pay-per-job, settled as jobs are executed.

RFQ quotes operator-signed, EIP-712 typed quotes accepted on-chain (see The RFQ System).

Fee split. Each service-fee payment is split across five recipients in basis points summing to 100%: operator 4000 (40%), developer 2000 (20%), staker 2000 (20%), protocol 1950 (19.5%), and keeper 50 (0.5%). The keeper's 0.5% is the rebate carved out of the protocol share that makes permissionless subscription billing worthwhile. The developer's 2000-bps share is also the source from which `BuybackBlueprintBase` funds token buybacks.

11.2 The RFQ System

Tangle uses request-for-quote (RFQ) for service pricing. Customers broadcast quote requests; operators return signed quotes; customers select and submit.

This accommodates operator heterogeneity, different hardware, locations, and capabilities command different prices.

11.3 Off-Chain Coordination

Quote propagation uses gossip protocols. Customers publish requests specifying blueprint, desired operators, parameters, and deadline. Requests propagate through the P2P network to registered operators.

Direct RPC queries bypass gossip for known operators. Each operator's endpoint is stored on-chain at registration.

Quote aggregators may collect quotes on behalf of customers, presenting curated options.

11.4 Failure Modes

Your blueprint should handle these scenarios:

Operator unavailable after quoting: Operator signs quote but becomes unavailable. Service creates but operator cannot fulfill. Handle through heartbeat monitoring and slashing.

Quote expiry during transaction: Quotes expire before transaction inclusion. Transaction fails with `QuoteExpired`. Set appropriate expiry times (not too short, not too long).

Capacity exhaustion: Operator signs multiple quotes exceeding capacity. Operator's responsibility to track outstanding quotes. Overcommitted operators face quality degradation and slashing.

Network partition: P2P partition prevents reaching some operators. Quote collection fails or produces incomplete results. Recovery occurs when partition heals.

11.5 Quote Serving Architecture

Operators run a separate **pricing engine** service alongside their blueprint. The pricing engine:

1. Receives quote requests via RPC or P2P gossip
2. Calculates prices based on resource requirements and operator pricing models
3. Signs the quote as EIP-712 typed data over the whole `QuoteDetails` struct
4. Returns the signed quote; spam resistance comes from the expiry timestamp and the operator's signature, with acceptance enforced on-chain (there is no proof-of-work)

The signed `QuoteDetails` struct contains: blueprint ID, TTL in *seconds* (enforced against `block.timestamp`; the struct field is named `ttlBlocks` for legacy reasons but the unit is seconds), total cost, timestamp, expiry, and security commitments (which assets back the operator's stake exposure). Because the EIP-712 signature covers the entire struct, any tampering with these fields invalidates the quote, and acceptance is verified on-chain at service creation.

Customers collect signed quotes from multiple operators, compare prices and terms, then submit their chosen quote on-chain to create a service.

11.6 Price Discovery

Prices converge as operators observe market conditions. On-chain service creation reveals accepted prices. Operators analyze historical prices and adjust quotes.

Quality differentiation prevents pure price competition. Operators with better reliability, faster hardware, or stronger security can command premiums.

12 Testing and Development

12.1 Local Testing

Mock producers generate test job calls without blockchain infrastructure:

```
#[tokio::test]
async fn test_square_handler() {
    let result = square(TangleArg(5)).await;
    assert_eq!(result.0, 25);
}
```

12.2 Integration Testing

Use local blockchain instances (Anvil, Hardhat) with deployed contracts:

```
#[tokio::test]
async fn test_full_flow() {
    let anvil = Anvil::new().spawn();
    let contracts = deploy_contracts(&anvil).await;

    // Submit job
    let job_id = submit_job(&contracts, 42).await;

    // Process with blueprint
    let runner = create_test_runner(&contracts).await;
    runner.process_pending().await;

    // Verify result
    let result = get_result(&contracts, job_id).await;
    assert_eq!(result, 42 * 42);
}
```

12.3 Testnet Deployment

Tangle testnet provides realistic pre-production validation with test tokens.

12.4 Observability

The SDK supports:

- Structured logging (configurable verbosity)
- Prometheus metrics export
- OpenTelemetry distributed tracing

13 P2P Networking

The SDK includes peer-to-peer networking built on libp2p, enabling operators to coordinate without centralized infrastructure.

13.1 Network Architecture

The networking layer provides:

NetworkService: Core service managing peer connections, message routing, and protocol handlers. Configured via **NetworkConfig** specifying listen addresses, bootstrap peers, and protocol settings.

NetworkServiceHandle: Thread-safe handle for interacting with the network from job handlers. Supports sending messages, subscribing to topics, and querying peer state.

13.2 Discovery Mechanisms

Kademlia DHT provides global peer discovery. Operators register in the distributed hash table, enabling any participant to find peers by their address or service membership. Bootstrap nodes seed initial peer connections.

mDNS enables local network discovery for development and testing. Operators on the same LAN discover each other automatically without external infrastructure.

PeerManager tracks peer metadata: public keys, addresses, connection state, and service registrations. The manager handles reconnection logic and peer scoring.

13.3 Messaging Protocols

Gossipsub provides efficient pub/sub messaging. Topics are identified by strings (e.g., service IDs). Messages propagate through the mesh network with configurable redundancy. Use cases include:

- Quote request broadcasting
- BLS signature aggregation
- Service announcements

Request-Response Protocol enables direct peer communication. The `BlueprintProtocol` defines typed messages:

- `InstanceMessageRequest`: Authenticated requests between operators
- `InstanceMessageResponse`: Responses with optional error handling

13.4 Authentication and Security

Peer authentication uses cryptographic handshakes:

AllowedKeys configures which peers may connect:

- `EvmAddresses`: Allow peers whose keys derive to specified addresses
- `InstancePublicKeys`: Allow specific public keys

Handshake protocol verifies peer identity before message exchange. Failed handshakes emit `HandshakeFailed` events for monitoring.

13.5 Network Events

The service emits typed events for application handling:

```
pub enum NetworkEvent<K: KeyType> {
    InstanceRequestInbound { peer: PeerId, request:
        InstanceMessageRequest<K> },
    InstanceResponseInbound { peer: PeerId, response:
        InstanceMessageResponse<K> },
    GossipReceived { source: PeerId, topic: String, message: Vec<u8> },
    PeerConnected(PeerId),
    PeerDisconnected(PeerId),
    HandshakeCompleted { peer: PeerId },
    HandshakeFailed { peer: PeerId, reason: String },
}
```

Job handlers can subscribe to these events for coordination logic.

13.6 BLS Signature Aggregation

The `agg-sig-gossip` feature provides the specialized gossip transport for the BLS signature aggregation flow described in the *BLS Aggregation* section: operators sign their results with BLS keys, broadcast the signatures over the per-job gossip topic, and an aggregator combines them once the threshold is met before submitting the aggregated result on-chain.

This pattern enables efficient multi-operator consensus without centralized coordination.

13.7 Custom Protocols

Blueprints can define custom protocols for domain-specific needs:

- Checkpoint synchronization for stateful services
- Shard assignment coordination for distributed processing
- Heartbeat protocols for liveness monitoring
- State machine replication for consensus-critical services

Custom protocols implement the libp2p `NetworkBehaviour` trait and integrate with the SDK's event system.

14 Deployment Checklist

Before deploying to production:

1. **Testing:** Unit tests, integration tests, testnet validation
2. **Error handling:** All failure modes covered with appropriate responses
3. **Monitoring:** Logging, metrics, and alerts configured
4. **Security:** Input validation, rate limiting, access controls
5. **Documentation:** Operator guide, API reference, troubleshooting
6. **Economics:** Slashing conditions, stake requirements, pricing model
7. **Verification:** Detection mechanisms appropriate to threat model

15 Operator Configuration

Operators configure their blueprint environment through command-line arguments, environment variables, or configuration files.

15.1 Core Settings

http_rpc_url EVM JSON-RPC endpoint for the host chain (local default: `http://127.0.0.1:8545`, the Anvil port; Base mainnet uses chainId 8453)

ws_rpc_url WebSocket EVM JSON-RPC endpoint for event subscriptions (local default: `ws://127.0.0.1:8545`)

keystore_uri Path to the keystore directory containing operator keys (default: `./keystore`)

data_dir Directory for blueprint-specific data storage (required)

protocol Target protocol: `tangle` (the only protocol supported at launch; `eigenlayer` and `symbiotic` are planned)

chain Network: `local_testnet`, `testnet`, or `mainnet`

15.2 Networking Settings

bootnodes Bootstrap peer addresses for P2P discovery (multiaddr format)

network__bind__port Port for P2P connections (default: random)

enable__mdns Enable local network discovery for development (default: false)

enable__kademlia Enable DHT-based global discovery (default: true in production)

target__peer__count Target number of peer connections (default: 24)

15.3 Example Configuration

Command-line invocation:

```
./my-blueprint run \
  --http-rpc-url https://rpc.tangle.tools \
  --ws-rpc-url wss://rpc.tangle.tools \
  --keystore-uri /var/lib/blueprint/keystore \
  --data-dir /var/lib/blueprint/data \
  --protocol tangle \
  --chain mainnet \
  --bootnodes /ip4/34.56.78.90/tcp/9000/p2p/12D3Koo... \
  --network-bind-port 9000 \
  --enable-kademlia
```

Environment variables use uppercase with underscores:

```
export HTTP_RPC_URL=https://rpc.tangle.tools
export WS_RPC_URL=wss://rpc.tangle.tools
export KEYSTORE_URI=/var/lib/blueprint/keystore
export DATA_DIR=/var/lib/blueprint/data
export PROTOCOL=tangle
export CHAIN=mainnet
```

15.4 Keystore Setup

The keystore contains cryptographic keys for operator identity and signing:

```
# Generate required keys
blueprint-cli key generate --scheme ecdsa --output ./keystore
blueprint-cli key generate --scheme ed25519 --output ./keystore
blueprint-cli key generate --scheme bls --output ./keystore

# List keys
blueprint-cli key list --keystore ./keystore
```

Required key types depend on the blueprint:

- **ECDSA**: Transaction signing and EVM address derivation
- **Ed25519**: P2P network identity and authentication
- **BLS**: Signature aggregation for multi-operator consensus

16 Resources

- **Blueprint SDK Repository**: <https://github.com/tangle-network/blueprint>
- **API Documentation**: <https://docs.tangle.tools/sdk>

- **Example Blueprints:** <https://github.com/tangle-network/blueprints>
- **Discord:** <https://discord.gg/tangle>